Gestion des évènements

32

- Se fait par l'intermédiaire des **listeners**.
  - Objets permettant de récupérer l'évènement et le traiter.
  - Interfaces dont chaque méthode correspond à une action qu'il est possible de faire à partir d'un évènement.
  - Par exemple, pour intercepter l'évènement « clic » sur un bouton, on applique l'interface ***View.OnClickListener*** sur ce bouton.
  - Cette interface contient la méthode **fun onClick(v: View?)**
    - Le paramètre de type View est la vue sur laquelle le clic a été effectué.
  - Il existe plusieurs méthodes pour utiliser ces Listeners.

Gestion des évènements – Listener en code Kotlin

33

Méthode 1

- Exemple: Clic sur un bouton
- Capturer tous les clics de bouton dans l'activité
  - La classe de l'activité doit implémenter ***View.OnClickListener***: Méthode par **HÉRITAGE**.

```
class MainActivity : AppCompatActivity(), View.OnClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val b = findViewById<Button>(R.id.button)
        b.setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        TODO( reason: "Not yet implemented")
    }
}
```

```
override fun onClick(v: View?) {
    TODO( reason: "not implemented")
}
```

Gestion des évènements – Listener en code Kotlin

34 Méthode 1

- Affichage d'un message pop-up.

```
override fun onClick(v: View?) {
    if (v != null) {
        Toast.makeText(v.context, text: "clic sur bouton",Toast.LENGTH_LONG).show()
    }
}
```



34

Gestion des évènements – Listener en code Kotlin

35 Méthode 1

- Un problème se pose !!!
  - Si on appuie sur un bouton quelconque, on rentre dans la même méthode onClick().
- Intérêt de la vue passée dans la méthode **onClick(v: View?)**
  - Permet de différencier les boutons.

```
override fun onClick(v: View?) {
        when (v?.id){
            R.id.button -> Toast.makeText(v.context, text: "clic sur bouton 1",Toast.LENGTH_LONG).show()
            R.id.button2 -> Toast.makeText(v.context, text: "clic sur bouton 2",Toast.LENGTH_LONG).show()
        }
}
```

35

Gestion des évènements – Listener en code Kotlin

36 Méthode 2

- Même exemple autrement

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val b = findViewById<Button>(R.id.button)
        b.setOnClickListener(object :View.OnClickListener{
            override fun onClick(v: View?) {
                Toast.makeText(v?.context,"clic sur bouton",Toast.LENGTH_LONG).show()
            }

        })
    }
}
```

- Méthode par **classe anonyme** ou **object expression** (instancier un objet qui dérive d'une superclasse ou implémente une interface).

36

Gestion des évènements – Listener en code Kotlin

37 Méthode 3

- Utilisation des lambdas

```
b.setOnClickListener(object :View.OnClickListener{
    override fun onClick(v: View?) {
        Toast.makeText(v?.context,"cl
    }
```

Convert to lambda

Convert to lambda Alt+Maj+Entrée More actions... Alt+Entrée

lambda

```
b.setOnClickListener { v:View? ->
    Toast.makeText(v?.context,"clic sur bouton",Toast.LENGTH_LONG).show() }
```

ou

```
b.setOnClickListener { v ->
    Toast.makeText(v?.context,"clic sur bouton",Toast.LENGTH_LONG).show() }
```

ou

```
b.setOnClickListener { it: View!
    Toast.makeText(this,"clic sur bouton",Toast.LENGTH_LONG).show() }
```

37

Gestion des évènements – Listener en code XML

38

- Définir le listener dans le code XML

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:onClick="buttonClick"
```

Résultat



- Gérer l'évènement dans le code Kotlin

```
class MainActivity:AppCompatActivity(){

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

    }

    fun buttonClick (view: View?){
        if (view != null) {
            Toast.makeText(view.context, text: "clic sur bouton",Toast.LENGTH_LONG).show()
        }
    }
}
```

38

Gestion des évènements

39

- **Exemple de passage d'une activité à une autre**
  - On crée un objet **Intent** (C'est un moyen pour faire communiquer des applications, des activités, etc.)

```
fun buttonClick (view: View?){
    val i = Intent(view?.context, Main2Activity::class.java)
    startActivity(i)
}
```

- Le premier argument représente le contexte de l'activité de départ (this / applicationContext / view.context).
- Le second représente l'activité d'arrivée/cible.
- La méthode « **startActivity** » lance l'activité sollicitée via l'intent créé précédemment.

39

Gestion des évènements

40

- Les boutons radios doivent toujours être placés dans un «RadioGroup».
  - Pour *garantir qu'un seul bouton puisse être sélectionné à la fois.*
  - Exemple: On cherche à avoir le comportement suivant:





40

Gestion des évènements

41

- Boutons Radios dans un « Radio Group »
  - Code XML

```
<RadioGroup
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/rg"
    android:layout_marginStart="87dp"
    android:layout_marginTop="72dp"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/button2">

    <RadioButton
        android:id="@+id/radioButton1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/femmme"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/button2" />
    <RadioButton
        android:id="@+id/radioButton2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/homme"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/radioButton1" />
</RadioGroup>
```

41

Gestion des évènements

42

- Boutons Radios dans un « Radio Group »
  - Code Kotlin avec **object expression**

```
class MainActivity:AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rg = findViewById<RadioGroup>(R.id.rg)
        rg.setOnCheckedChangeListener(object:RadioGroup.OnCheckedChangeListener{
            override fun onCheckedChanged(group: RadioGroup?, checkedId: Int) {
                when (checkedId) {
                    R.id.radioButton1 -> Toast.makeText(applicationContext,  text: "Femme", Toast.LENGTH_LONG).show()
                    R.id.radioButton2 -> Toast.makeText(applicationContext,  text: "Homme", Toast.LENGTH_LONG).show()
                }
            }
        })
    }
}
```

42

Gestion des évènements

43

- Boutons Radios dans un « Radio Group »
  - Code Kotlin avec **lambda**

```
class MainActivity:AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val rg = findViewById<RadioGroup>(R.id.rg)
        rg.setOnCheckedChangeListener { radioGroup: RadioGroup, i: Int ->
            when (i) {
                R.id.radioButton1 -> Toast.makeText( context: this,  text: "Femme", Toast.LENGTH_LONG).show()
                R.id.radioButton2 -> Toast.makeText( context: this,  text: "Homme", Toast.LENGTH_LONG).show()
            }
        }
    }
}
```

43

Layouts

44

- Les layouts se trouvent dans le répertoire res/layout
  - Utilisables dans le projet avec la méthode **setContentView()**
    - setContentView(R.layout.activity_main);

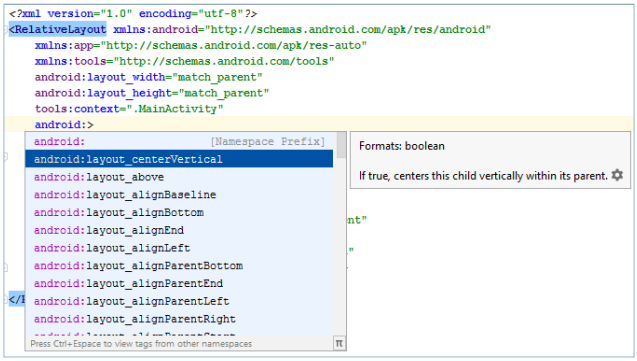
```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

44

Layouts

45

- Leurs définitions se font principalement en XML (interfaces statiques).
  - La définition en Java ou en Kotlin convient aux interfaces dynamiques.
  - La combinaison des deux types est possible.



45

Layouts

46

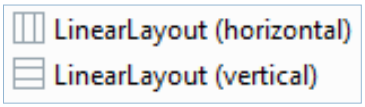
- Un layout reste toujours modifiable à travers le code.
- Pour cela, si nécessaire, il devient important de spécifier un ***id*** dans la définition XML du gabarit.
  - **android:id**= "**@+id**/idElem"

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:id="@+id/layout"
    android:orientation="horizontal">
</LinearLayout>
```

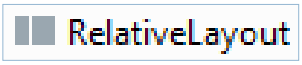

Layouts – Types

47

- **LinearLayout**: organise les différents éléments de l'interface sur:
  - une ligne (*Horizontal*: Les élts sont placés de gauche à droite)
  - une colonne (*Vertical*: Les élts. sont empilés du haut vers le bas).



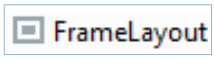
- **RelativeLayout**: permet de placer un élément en fonction du layout parent et par rapport à la position des autres éléments.



Layouts – Types

48

- **FrameLayout**: utilisé pour afficher une seule vue à la fois.
  - Il sert principalement à superposer des vues (View) les unes sur les autres, comme des calques (layers).
  - Peut contenir de nombreux widgets, mais un seul apparaît.
  - Il n'organise pas automatiquement ses enfants (comme le fait LinearLayout ou RelativeLayout).
  - Le premier élément ajouté s'affiche en dessous, et les suivants sont dessus (ordre de déclaration).
  - Chaque vue est positionnée **en haut à gauche** par défaut.
  - Il est souvent utilisé pour afficher un seul élément ou faire des superpositions.
  - Permet de jouer sur leur visibilité.



48

Layouts – Types

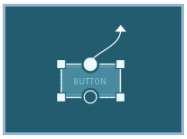
49

- **ConstraintLayout**: vient d'un nouveau package de widgets

```
import androidx.constraintlayout.widget.ConstraintLayout
```

- Il est comme le RelativeLayout (position par rapport au layout parent ou aux autres éléments), mais c'est plus flexible.
  - Au moins deux contraintes doivent être définies (une verticale et une horizontale) pour chaque vue.
  - Chaque contrainte représente une connexion ou un alignement.
  - Si aucune contrainte n'est définie pour une vue, elle sera affichée à la position (0,0) (coin gauche en haut).

➔ Le plus utilisé aujourd'hui, très flexible, nécessite des contraintes horizontales/verticales.



49

Configuration – ConstraintLayout

50

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

- **android:layout_width** et **android:layout_height**
  - Ces deux propriétés peuvent prendre 3 types de valeur:
    - ***match_parent***: le layout occupe tout l'espace disponible chez son conteneur parent (écran ou autre).
    - ***wrap_content***: le composant prend la taille de son contenu.
    - **Une taille fixe**: par exemple 100dp.

50

Configuration – ConstraintLayout

51

- **Taille des éléments (Unités) :** valable pour tous les layouts et tous les objets d'interface.
  - dp ou dip (Density-independant Pixel) : il s'agit d'une unité qui est indépendante de la densité de l'écran.
    - Il est déconseillé d'utiliser le pixel (px) ou le millimètre (mm), car ces unités varient d'un écran à l'autre.
    - 200 dp: le widget aura toujours une dimension proportionnelle quelle que soit la taille de l'écran.
    - 200 mm: le widget sera suffisamment grand pour un grand écran et énorme pour un petit écran.
  - sp (Scale-independant Pixel) : unité plus adaptée pour définir la taille d'une police de caractères.
    - Elle respecte le même principe que dp.

51

Configuration – ConstraintLayout

52

- **ConstraintLayout : Les « guidelines »**
  - Une Guideline est un composant virtuel (qui n'apparait pas sur l'écran) représentant une ligne de référence, verticale ou horizontale, sur laquelle les vues pourront venir s'ancrer.
  - Une *Guideline* peut être placée de 3 façons :
    - Distance en « *dp*» depuis le haut (pour une orientation horizontale) ou du côté gauche (pour une orientation verticale)

```
app:layout_constraintGuide_begin="100dp"
```

    - Distance en « *dp*» depuis le bas (orientation horizontale) ou du côté droit (orientation verticale)

```
app:layout_constraintGuide_end="200dp"
```

    - En pourcentage sur l'axe vertical ou horizontal

```
app:layout_constraintGuide_percent="0.25"
```

52

Configuration – ConstraintLayout

53

- **ConstraintLayout : Les « groups »**
  - La contrainte « *Groups* » permet de rassembler, de manière transparente sur l'interface, des *vues* sous un identifiant commun.
  - Ceci servira à appliquer la même chose à un ensemble d'éléments graphiques en même temps.
    - La visibilité, une animation particulière, etc.

```
<androidx.constraintlayout.widget.Group
    android:id="@+id/grp"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:constraint_referenced_ids="tvValue2, editValue2"/>
```

```
grp.visibility=View.INVISIBLE
```

Ceci rend invisibles à la fois les vues ayant les ids « tvValue2 » et « editValue2 »

53

Pour Finir…

54

- L'IHM (UI) est d'une grande importance pour les applications Android (c'est ce que l'utilisateur voit et utilise).
  - Il faut bien choisir les vues, leurs dispositions, leurs propriétés, etc.
  - Sur un écran à taille limitée, organiser l'IHM n'est pas forcément une chose facile.
    - Trouver des moyens pour la bonne organisation de l'IHM (fragments, onglets, etc.)
- Toute activité suit un cycle de vie et il faut bien gérer les différentes phases.

54

Récap.

- UI Android = fichiers XML reliés au code via R.**layout**.
- **Widgets** = éléments visibles, **Layouts** = organisation.
- Plusieurs façons de gérer les vues : **findViewById**(), **View Binding** (méthode moderne).
- Événements = **listeners** (héritage, classes anonymes, lambdas).
- Navigation entre écrans = **Intent**.
- Bien choisir le bon layout (**ConstraintLayout** recommandé).
- Toujours penser à l'expérience utilisateur → simplicité, lisibilité.

55